
Lab 04

Objective:

The objective of this lab is to learn how to implement self organizing maps in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement SOM model in python.
- Implement training algorithm to train SOM using a dataset.

Instructor Note:

As pre-lab activity, read and implement the following tutorial:

<https://towardsdatascience.com/understanding-self-organising-map-neural-network-with-python-code-7a77f501e985>

8) Useful Concepts

Self-organizing maps: The Self-Organising Map (SOM) is an unsupervised machine learning algorithm introduced by Teuvo Kohonen in the 1980s. As the name suggests, the map organises itself without any instruction from others. It is a brain-inspired model. A different area of the cerebral cortex in our brain is responsible for specific activities. A sensory input like vision, hearing, smell, and taste is mapped to neurons of a corresponding cortex area via synapses in a self-organising way. It is also known that the neurons with similar output are in proximity. SOM is trained through a competitive neural network, a single-layer feed-forward network that resembles these brain mechanisms.

The SOM's algorithm is relatively simple, but there may be some confusion at first sight and difficulties figuring out how to apply it in practice. It may be because SOM can be understood from multiple perspectives. It is like Principal Component Analysis (PCA) for dimensionality reduction and visualisation. SOM can also be considered a type of manifold learning that handles non-linear dimensionality reduction. SOM is also used in data mining for its vector quantisation property. The train can represent high-dimensional observable data onto lower dimension latent space, typically on a 2D square grid, while preserving the topology of the original input space. But the map can also be used for projecting new data points and seeing which cluster belongs to the map, too.

Architecture and Learning Algorithm: The neural network of the Self-Organising Map has one input layer and one output layer. The second layer typically consists of a two-dimensional lattice of $m \times n$ neurons. Each neuron at the map layer is densely connected to all neurons in the input layer, possessing different weight values.

In competitive learning, neurons in the output layer compete among themselves to be activated. The winning neuron of the competition is the only one that can be fired; hence it is called the winner-takes-all neuron. In SOM, the competitive process is to search for the most similar neuron with the input pattern; the winner is called Best Matching Unit (BMU).

Similarity as the winning criterion can be measured in several ways. The most common measure employed is Euclidean distance. The neuron with the shortest distance from the input signal becomes the BMU.

In SOM, learning is not only for the winner but also for the neurons in physical proximity to it on the map. The BMU shares the privilege with its neighbours to learn together. The definition of the neighbourhood is determined by the network designer, and the optimal proximity depends on other hyperparameters. If the neighbourhood range is too small, the trained model will suffer from overfitting, and there is a risk of having some dead neurons that never have an opportunity to change.

In the adaptation phase, the BMU and its neighbours adjust their weight. The effect of the learning is to move the weight of the winning and neighbouring neurons closer to the input pattern. As an example, if the input signal is blue colour and the BMU neuron is light blue colour, the winner becomes somewhat bluer than light blue. If the neighbours are yellow, they are added a little bit of blueness to their current colour.

The Self-Organising Map learning algorithm (online learning) can be described in the following 4 steps.

1. Initialisation
Weights of neurons in the map layer are initialised.
 2. Competitive process
Select one input sample and search the best matching unit among all neurons in $n \times m$ grid using distance measures.
 3. Cooperative process
Find the proximity neurons of BMU by neighbourhood function.
 4. Adaptation process
Update the BMU and neighbours' weights by shifting the values towards the input pattern.
- If the maximum count of training iteration is reached, exit. If not, increment the iteration count by 1 and repeat the process from 2.

9) Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
1	10	Medium	CLO-3
2	10	Medium	CLO-3
3	10	Medium	CLO-3
4	10	Medium	CLO-3
5	10	Medium	CLO-3

In this lab, the SOM is trained with the Banknote Authentication data set available at the UCI ML Repository website. The data file contains 1372 rows, each with 4 features and 1 label. All 4 features are numerical values without null. The label is a binary integer value.

Firstly, we randomly split the data into training and testing data. We use all 4 features to train the SOM using an online training algorithm. Then, we evaluate the trained SOM by visualising the map using the label of the training data. Finally, we can use the test data to predict the label using the trained map.

Thus, we can demonstrate that a SOM trained in an unsupervised manner can be applied for classification using a labelled dataset.

The tutorial is available at: <https://towardsdatascience.com/understanding-self-organising-map-neural-network-with-python-code-7a77f501e985>

Activity 1: Data pre-processing

Solution:

```
import numpy as np

from sklearn import datasets

from sklearn.preprocessing import MinMaxScaler

# Load Iris dataset

data = datasets.load_iris()

X = data.data

# Normalize the dataset

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)
```

Activity 2: Implementing SOM

Solution:

```
class SOM:

    def __init__(self, m, n, dim, learning_rate=0.5, sigma=None):

        self.m = m
```

```
self.n = n

self.dim = dim

self.learning_rate = learning_rate

self.sigma = sigma if sigma else max(m, n) / 2.0

self.weights = np.random.rand(m, n, dim)

def _find_bmu(self, x):

    distances = np.linalg.norm(self.weights - x, axis=-1)

    bmu_index = np.unravel_index(np.argmin(distances), (self.m, self.n))

    return bmu_index

def _update_weights(self, x, bmu, iteration, max_iter):

    learning_rate = self.learning_rate * np.exp(-iteration / max_iter)

    sigma = self.sigma * np.exp(-iteration / max_iter)

    for i in range(self.m):

        for j in range(self.n):

            w = self.weights[i, j]

            distance = np.linalg.norm(np.array([i, j]) - np.array(bmu))
```

```
        if distance < sigma:

            influence = np.exp(-distance / (2 * (sigma ** 2)))

            self.weights[i, j] += influence * learning_rate * (x - w)

    def train(self, X, num_iterations):

        for iteration in range(num_iterations):

            for x in X:

                bmu = self._find_bmu(x)

                self._update_weights(x, bmu, iteration, num_iterations)
```

Activity 3: Training the SOM

Solution:

```
som = SOM(m=10, n=10, dim=4, learning_rate=0.5)

som.train(X_scaled, num_iterations=100)
```

Activity 4: Visualization

Solution:

```
import matplotlib.pyplot as plt

def plot_som(som):

    plt.figure(figsize=(10, 10))
```

```
for i in range(som.m):

    for j in range(som.n):

        plt.scatter(j, i, c=som.weights[i, j], marker='o', s=300, edgecolor='k')

plt.title('SOM Grid')

plt.show()

plot_som(som)
```

Activity 5: Clustering with SOM

Solution:

```
def get_cluster_labels(som, X):

    cluster_labels = []

    for x in X:

        bmu = som._find_bmu(x)

        cluster_labels.append(bmu)

    return np.array(cluster_labels)

cluster_labels = get_cluster_labels(som, X_scaled)

print(cluster_labels)
```

10) Graded Lab Tasks

The purpose of this lab task is to help students understand and implement self-organizing maps.

Lab Task 1: Build and Train Self-organizing Map in Python

Download a dataset from Kaggle or UCI machine learning repository and train SOMs algorithm on the dataset.

Instructions for Submission:

1. Submit a Jupyter Notebook or Python script with the complete implementation.
2. Include comments in the code to explain each step.
3. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics
 - Observations and insights from the results